

AWS Lambda Container Image Deployment with Amazon ECR

By

Esmeralda C. Cabrera Ventura

Project Overview

AWS Lambda traditionally required functions to be packaged as ZIP archives, which could limit flexibility for teams that prefer container-based development workflows. With AWS Lambda's support for OCI-compliant container images, developers can now combine serverless execution with Docker-based packaging, gaining better control over dependencies, runtime behavior, and build reproducibility.

This project demonstrates an end-to-end containerized serverless deployment by building a Python web application as a Docker image, storing it in Amazon Elastic Container Registry (ECR), and deploying it as an AWS Lambda function.

Situation / Problem Statement

Modern development teams often standardize on containers to ensure consistent builds across environments. However, they still want to benefit from:

- Event-driven execution
- Automatic scaling
- Pay-per-use pricing
- Minimal infrastructure management

The challenge is to bridge containerized development with a serverless runtime, without managing long-running servers or orchestration platforms.

Arrived Solution

The solution uses AWS Lambda container image support:

1. A Python application is packaged as a Docker image using an Alpine Linux base.
2. The image is built on an EC2 instance acting as a temporary build host.
3. The container image is stored in a private Amazon ECR repository.
4. AWS Lambda pulls the image directly from ECR and executes it as a function.
5. The deployment is validated using a test invocation.

This approach preserves container tooling while eliminating the need to manage EC2 instances in production.

Architecture Design

High-Level Architecture Flow

Developer / Build Host (EC2)



Docker Build (OCI Image)



Amazon ECR (Private Repository)

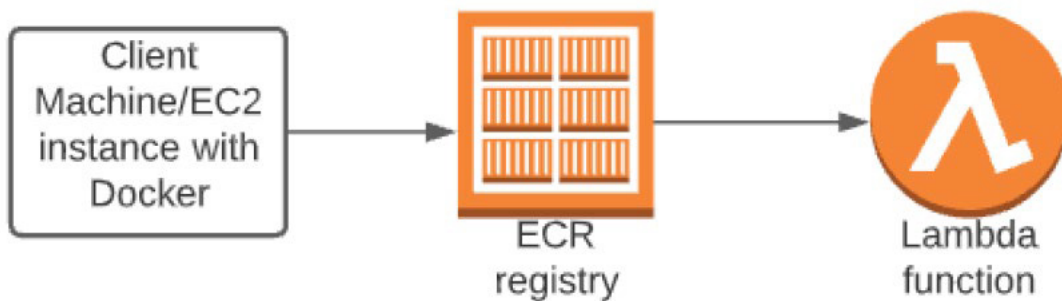


AWS Lambda (Container Image Runtime)



Function Invocation (Test Event)

Diagram Architecture



AWS Services Used

- **EC2 (Ubuntu 22.04)** – temporary build environment for Docker
- **Docker** – containerization of the application
- **Amazon ECR** – private container image registry
- **AWS Lambda** – serverless execution of the container image
- **IAM Roles** – secure permissions for EC2 and Lambda

This design cleanly separates build responsibilities from runtime execution, which mirrors real-world CI/CD patterns.

Step-by-Step Implementation Guide

Step 1: Docker Image Creation

Objective: Package the application as an OCI-compliant container image.

- Launched an Ubuntu 22.04 EC2 instance in the default VPC.
- Attached the IAM role LabInstanceProfile (or an equivalent role with ECR access).
- Uploaded the provided OCI.zip file to the instance using scp.
- Installed Docker and supporting tools using the provided installation script.
- Installed and configured AWS CLI:
 - Region: us-east-1
 - Output format: json
- Built the Docker image using the provided Dockerfile:
`docker build -t lambda_ecr .`
- Verified that the image was successfully created using the command:
`docker images`

Result:

A locally built Docker image containing the Python application, ready for registry upload.

Docker Image Creation

```
ubuntu@ip-172-31-29-121: ~$ docker build -t lambda_ecr .
[+] Building 11.5s (8/8) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 142B                             0.0s
=> [internal] load metadata for docker.io/library/python:alpine 0.5s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 2.82kB                               0.0s
=> [1/3] FROM docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 1.6s
=> => resolve docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 0.0s
=> => sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 456.92kB / 456.92kB 0.1s
=> => sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 13.23MB / 13.23MB 0.2s
=> => sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 250B / 250B 0.2s
=> => sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 10.29kB / 10.29kB 0.0s
=> => sha256:da0af7e1f95f7612c15858b627b151edde12a97b6ae60731a82c4849bd9c5b2c 1.73kB / 1.73kB 0.0s
=> => sha256:24ef78ff5ffe3f0b75661012741566f58f76e8109513be875a237d0a7b19b073 4.62kB / 4.62kB 0.0s
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s
=> => extracting sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 0.1s
=> => extracting sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 0.8s
=> => extracting sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 0.0s
=> [2/3] COPY ./content . 0.1s
=> [3/3] RUN pip install -r requirements.txt 8.1s
=> => exporting to image 1.0s
=> => exporting layers 1.0s
=> => writing image sha256:23f4d9cf57b91f8656cbc72f53570b6628355dad3e72e5467b46993e0e013341 0.0s
=> => naming to docker.io/library/lambda_ecr 0.0s

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)
ubuntu@ip-172-31-29-121:~$
```

Docker Images List

```
ubuntu@ip-172-31-29-121: ~$ docker build -t lambda_ecr .
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 142B                             0.0s
=> [internal] load metadata for docker.io/library/python:alpine 0.5s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 2.82kB                               0.0s
=> [1/3] FROM docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 1.6s
=> => resolve docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 0.0s
=> => sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 456.92kB / 456.92kB 0.1s
=> => sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 13.23MB / 13.23MB 0.2s
=> => sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 250B / 250B 0.2s
=> => sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 10.29kB / 10.29kB 0.0s
=> => sha256:da0af7e1f95f7612c15858b627b151edde12a97b6ae60731a82c4849bd9c5b2c 1.73kB / 1.73kB 0.0s
=> => sha256:24ef78ff5ffe3f0b75661012741566f58f76e8109513be875a237d0a7b19b073 4.62kB / 4.62kB 0.0s
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s
=> => extracting sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 0.1s
=> => extracting sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 0.8s
=> => extracting sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 0.0s
=> [2/3] COPY ./content . 0.1s
=> [3/3] RUN pip install -r requirements.txt 8.1s
=> => exporting to image 1.0s
=> => exporting layers 1.0s
=> => writing image sha256:23f4d9cf57b91f8656cbc72f53570b6628355dad3e72e5467b46993e0e013341 0.0s
=> => naming to docker.io/library/lambda_ecr 0.0s

1 warning found (use docker --debug to expand):
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)
ubuntu@ip-172-31-29-121:~$ docker images | grep lambda_ecr
lambda_ecr latest 23f4d9cf57b9 46 seconds ago 98.4MB
ubuntu@ip-172-31-29-121:~$
```

Step 2: Create Amazon ECR Repository and Upload Image

Objective: Store the container image in a managed, secure registry.

1. Navigate to Amazon ECR in the AWS Console.
2. Create a private repository named `lambda_ecr`.
3. Open View push commands for the repository.
4. Authenticate Docker to ECR using AWS CLI.
5. Tag the local Docker image with the ECR repository URI.
6. Push the image to ECR.

To simplify commands, a shell variable was used to store the AWS account ID:

```
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
```

Result:

The Docker image is successfully stored in Amazon ECR and available for Lambda consumption.

Amazon ECR Repository Creation

The screenshot displays the Amazon ECR console interface. At the top, a green banner indicates 'Successfully created lambda_ecr'. Below this, the 'Private repositories (1)' section shows a table with the following data:

Repository name	URI	Created at	Tag immutability	Encryption type
lambda_ecr	524248575754.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr	November 04, 2025, 21:06:19 (UTC-08)	Mutable	AES-256

The left sidebar shows the navigation menu with 'Private registry' selected. The top navigation bar includes the AWS logo, search bar, and account information (United States (N. Virginia), Account ID: 5242-4857-5754).

ECR View Push Commands

```
ubuntu@ip-172-31-29-121: ~  
=> => sha256:da0af7e1f95f7612c15858b627b151edde12a97b6ae60731a82c4849bd9c5b2c 1.73kB / 1.73kB 0.0s  
=> => sha256:24ef78ff5ffe3f0b75661012741566f58f76e8109513be875a237d0a7b19b073 4.62kB / 4.62kB 0.0s  
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s  
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s  
=> => extracting sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 0.1s  
=> => extracting sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 0.8s  
=> => extracting sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 0.0s  
=> [2/3] COPY ./content . 0.1s  
=> [3/3] RUN pip install -r requirements.txt 8.1s  
=> exporting to image 1.0s  
=> exporting layers 1.0s  
=> => writing image sha256:23f4d9cf57b91f8656cbc72f53570b6628355dad3e72e5467b46993e0e013341 0.0s  
=> => naming to docker.io/library/lambda_ecr 0.0s  
  
1 warning found (use docker --debug to expand):  
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)  
ubuntu@ip-172-31-29-121:~$ docker images | grep lambda_ecr  
lambda_ecr latest 23f4d9cf57b9 46 seconds ago 98.4MB  
ubuntu@ip-172-31-29-121:~$ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)  
ubuntu@ip-172-31-29-121:~$ echo $ACCOUNT_ID  
524248575754  
ubuntu@ip-172-31-29-121:~$ docker tag lambda_ecr:latest ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr:latest  
ubuntu@ip-172-31-29-121:~$ docker push ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr:latest
```

AWS Account Number Stored in \$(ACCOUNT_ID)

```
ubuntu@ip-172-31-29-121: ~  
=> [internal] load .dockerignore 0.0s  
=> => transferring context: 28 0.0s  
=> [internal] load build context 0.0s  
=> => transferring context: 2.82kB 0.0s  
=> [1/3] FROM docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 1.6s  
=> => resolve docker.io/library/python:alpine@sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 0.0s  
=> => sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 456.92kB / 456.92kB 0.1s  
=> => sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 13.23MB / 13.23MB 0.2s  
=> => sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 250B / 250B 0.2s  
=> => sha256:8373231e1e906ddfb457748bfc032c4c06ada8c759b7b62d9c73ec2a3c56e710 10.29kB / 10.29kB 0.0s  
=> => sha256:da0af7e1f95f7612c15858b627b151edde12a97b6ae60731a82c4849bd9c5b2c 1.73kB / 1.73kB 0.0s  
=> => sha256:24ef78ff5ffe3f0b75661012741566f58f76e8109513be875a237d0a7b19b073 4.62kB / 4.62kB 0.0s  
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s  
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s  
=> => extracting sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 0.1s  
=> => extracting sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 0.8s  
=> => extracting sha256:13839c1b2c2fc819a0ebcba5bbfcb61b3d29976827c3c86062f87186660baa70 0.0s  
=> [2/3] COPY ./content . 0.1s  
=> [3/3] RUN pip install -r requirements.txt 8.1s  
=> exporting to image 1.0s  
=> exporting layers 1.0s  
=> => writing image sha256:23f4d9cf57b91f8656cbc72f53570b6628355dad3e72e5467b46993e0e013341 0.0s  
=> => naming to docker.io/library/lambda_ecr 0.0s  
  
1 warning found (use docker --debug to expand):  
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)  
ubuntu@ip-172-31-29-121:~$ docker images | grep lambda_ecr  
lambda_ecr latest 23f4d9cf57b9 46 seconds ago 98.4MB  
ubuntu@ip-172-31-29-121:~$ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)  
ubuntu@ip-172-31-29-121:~$ echo $ACCOUNT_ID  
524248575754  
ubuntu@ip-172-31-29-121:~$
```

ECR Login Succeeded

```
ubuntu@ip-172-31-29-121: ~  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
unzip is already the newest version (6.0-2ubuntu4.1).  
curl is already the newest version (8.5.0-2ubuntu10.6).  
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.  
ubuntu@ip-172-31-29-121:~$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"  
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
                                 Dload  Upload  Total   Spent    Left   Speed  
100 59.3M  100 59.3M    0     0  207M      0 --:--:-- --:--:-- --:--:-- 208M  
ubuntu@ip-172-31-29-121:~$ unzip -q awscliv2.zip  
ubuntu@ip-172-31-29-121:~$ sudo ./aws/install  
You can now run: /usr/local/bin/aws --version  
ubuntu@ip-172-31-29-121:~$ aws --version  
aws-cli/2.31.29 Python/3.13.9 Linux/6.14.0-1015-aws exe/x86_64.ubuntu.24  
ubuntu@ip-172-31-29-121:~$ aws configure  
AWS Access Key ID [None]:  
AWS Secret Access Key [None]:  
Default region name [None]: us-east-1  
Default output format [None]: json  
ubuntu@ip-172-31-29-121:~$ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)  
ubuntu@ip-172-31-29-121:~$ aws ecr get-login-password --region us-east-1 \  
> | docker login --username AWS --password-stdin ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com  
  
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
ubuntu@ip-172-31-29-121:~$
```

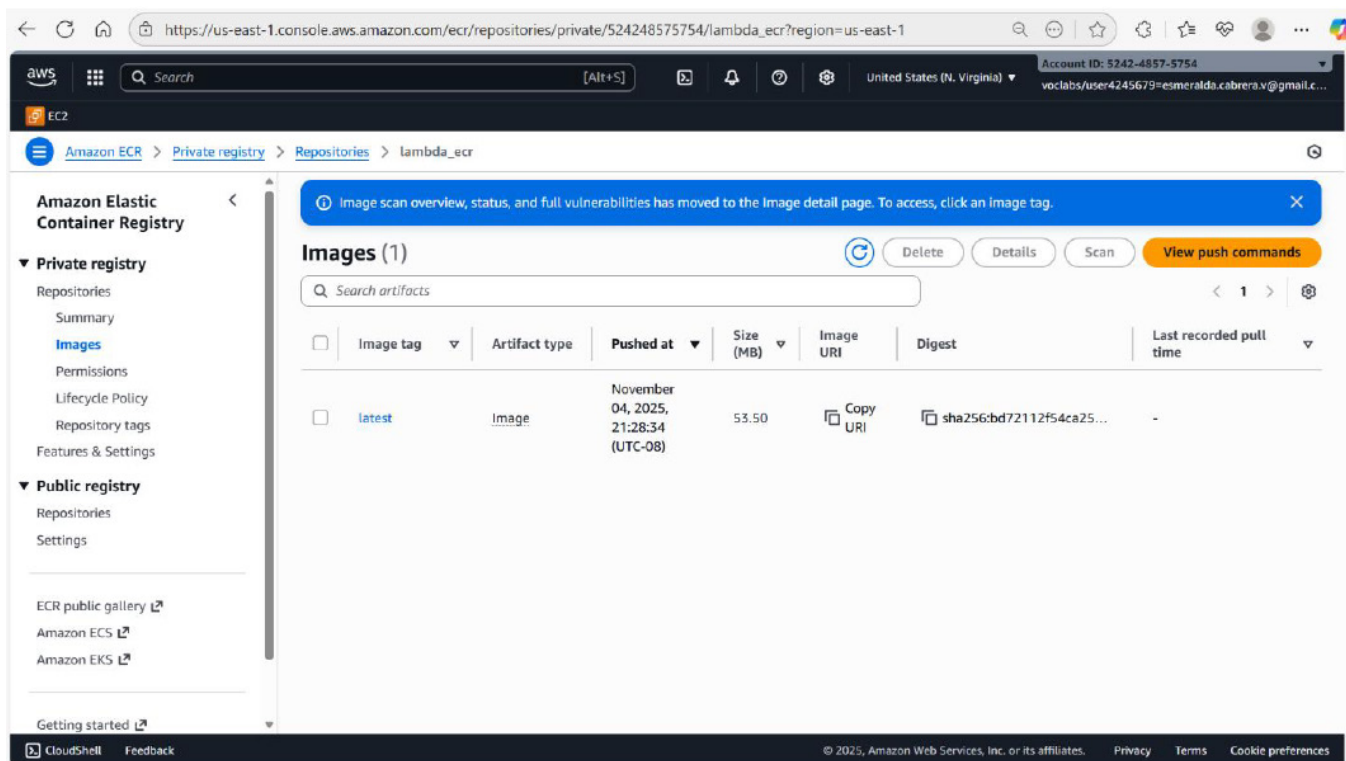
Tagging of Docker Images

```
ubuntu@ip-172-31-29-121: ~  
curl is already the newest version (8.5.0-2ubuntu10.6).  
0 upgraded, 0 newly installed, 0 to remove and 9 not upgraded.  
ubuntu@ip-172-31-29-121:~$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"  
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
                                 Dload  Upload  Total   Spent    Left   Speed  
100 59.3M  100 59.3M    0     0  207M      0 --:--:-- --:--:-- --:--:-- 208M  
ubuntu@ip-172-31-29-121:~$ unzip -q awscliv2.zip  
ubuntu@ip-172-31-29-121:~$ sudo ./aws/install  
You can now run: /usr/local/bin/aws --version  
ubuntu@ip-172-31-29-121:~$ aws --version  
aws-cli/2.31.29 Python/3.13.9 Linux/6.14.0-1015-aws exe/x86_64.ubuntu.24  
ubuntu@ip-172-31-29-121:~$ aws configure  
AWS Access Key ID [None]:  
AWS Secret Access Key [None]:  
Default region name [None]: us-east-1  
Default output format [None]: json  
ubuntu@ip-172-31-29-121:~$ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)  
ubuntu@ip-172-31-29-121:~$ aws ecr get-login-password --region us-east-1 \  
> | docker login --username AWS --password-stdin ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com  
  
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
ubuntu@ip-172-31-29-121:~$ docker tag lambda_ecr:latest ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr:latest  
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fva  
r%2Frun%2Fdocker.sock/v1.51/images/lambda_ecr:latest/tag?repo=524248575754.dkr.ecr.us-east-1.amazonaws.com%2Flambda_ecr&  
tag=latest": dial unix /var/run/docker.sock: connect: permission denied  
ubuntu@ip-172-31-29-121:~$
```


Docker Image Pushed

```
ubuntu@ip-172-31-29-121: ~  
=> sha256:da0af7e1f95f7612c15858b627b151edde12a97b6ae60731a82c4849bd9c5b2c 1.73kB / 1.73kB 0.0s  
=> sha256:24ef78ff5ffe3f0b75661012741566f58f76e8109513be875a237d0a7b19b073 4.62kB / 4.62kB 0.0s  
=> sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB 0.1s  
=> extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 0.2s  
=> extracting sha256:86d375910f52b7c453db7c1e2135c1536210eb84825e8505b1c9e5a34faaf57a 0.1s  
=> extracting sha256:facdf984afd18f7f52714321a1e8915796191dd98798aef5d1f77a85507e830d 0.8s  
=> extracting sha256:13839c1b2c2fc819a0ebc5b5bbfcb61b3d29976827c3c86062f87186660baa70 0.0s  
=> [2/3] COPY ./content . 0.1s  
=> [3/3] RUN pip install -r requirements.txt 8.1s  
=> exporting to image 1.0s  
=> exporting layers 1.0s  
=> writing image sha256:23f4d9cf57b91f8656cbc72f53570b6628355dad3e72e5467b46993e0e013341 0.0s  
=> naming to docker.io/library/lambda_ecr 0.0s  
  
1 warning found (use docker --debug to expand):  
- JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)  
ubuntu@ip-172-31-29-121:~$ docker images | grep lambda_ecr  
lambda_ecr latest 23f4d9cf57b9 46 seconds ago 98.4MB  
ubuntu@ip-172-31-29-121:~$ ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)  
ubuntu@ip-172-31-29-121:~$ echo $ACCOUNT_ID  
524248575754  
ubuntu@ip-172-31-29-121:~$ docker tag lambda_ecr:latest ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr:latest  
ubuntu@ip-172-31-29-121:~$ docker push ${ACCOUNT_ID}.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr:latest  
The push refers to repository [524248575754.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr]  
a2d6dbb0bd6c: Pushed  
ef08f802a15b: Pushed  
032805dd327f: Pushed  
3127c943e89e: Pushed  
9c9b4a941ab5: Pushed  
256f393e029f: Pushed  
latest: digest: sha256:bd72112f54ca254d4eb4aa1f24f68b9332eeb05d78b15016cdf3a78fc60a1e88 size: 1576  
ubuntu@ip-172-31-29-121:~$
```

ECR Image Successfully Uploaded to Repo



Step 3: Create AWS Lambda Function from Container Image

Objective: Deploy and validate the container image using Lambda.

- Navigated to AWS Lambda in the AWS Console.
- Selected Create function.
- Chose Container image as the deployment option.
- Browsed and selected the image from the ECR repository.
- Assigned the existing IAM role LabRole.
- Created the function and waited for provisioning to complete.
- Ran the default Hello World test event.

Result:

The Lambda function executed successfully using the container image pulled from ECR.

Lambda Container Image Selection

The screenshot shows the AWS Lambda console 'Create function' page for the 'us-east-1' region. The page is titled 'Create function' and includes the following sections:

- Function name:** A text input field containing 'lambda-ecr-test'. Below it, a note states: 'Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphen (-), and underscores (_).'.
- Container image URI:** A text input field containing '524248575754.dkr.ecr.us-east-1.amazonaws.com/lambda_ecr@sha256:bd72112f54ca254d4eb4aa1f24f68b9532eeb05d78b15016cdf5a78fc60a1e88'. To the right is a 'Browse images' button. Below the field, a note states: 'Requires a valid Amazon ECR image URI.'
- Container image overrides:** A section with a plus icon and the title 'Container image overrides'.
- Architecture:** A section with the title 'Architecture' and a note: 'Choose the instruction set architecture you want for your function code.' It has two radio buttons: 'arm64' (unselected) and 'x86_64' (selected).
- Permissions:** A section with the title 'Permissions' and a note: 'By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.'
- Change default execution role:** A section with a dropdown arrow and the title 'Change default execution role'. It contains three radio buttons: 'Create a new role with basic Lambda permissions' (unselected), 'Use an existing role' (selected), and 'Create a new role from AWS policy templates' (unselected).
- Existing role:** A section with a note: 'Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.' It has a dropdown menu with 'LabRole' selected. To the right is a 'View the LabRole role' button.

The bottom of the page shows the 'CloudShell' tab and the 'Feedback' link. The footer contains the copyright notice '© 2025, Amazon Web Services, Inc. or its affiliates.' and links for 'Privacy', 'Terms', and 'Cookie preferences'.

Lambda Execution Role Selection

The screenshot shows the AWS Lambda console 'Create function' page. The breadcrumb navigation is 'Lambda > Functions > Create function'. The 'Permissions' section is active, with a sub-header 'Change default execution role'. Under 'Execution role', the option 'Use an existing role' is selected. In the 'Existing role' dropdown, 'LabRole' is chosen. A green success message at the top states: 'Successfully created the function lambda-ecr-test. You can now change its code and configuration. To invoke your function with a test event, choose "Test".'

Permissions [Info](#)

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

- ☐ Create a new role with basic Lambda permissions
- ☒ Use an existing role
- ☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

LabRole [View the LabRole role on the IAM console.](#)

► **Additional configurations**

[Cancel](#) [Create function](#)

Lambda Function Creation

The screenshot shows the AWS Lambda console 'Function overview' page for the function 'lambda-ecr-test'. The breadcrumb navigation is 'Lambda > Functions > lambda-ecr-test'. The 'Function overview' section is active, showing a diagram of the function and buttons for 'Add trigger' and 'Add destination'. The right sidebar displays the function's details: Description, Last modified (1 minute ago), Function ARN, and Function URL. A green success message at the top states: 'Successfully created the function lambda-ecr-test. You can now change its code and configuration. To invoke your function with a test event, choose "Test".'

lambda-ecr-test [Throttle](#) [Copy ARN](#) [Actions](#)

▼ **Function overview** [Info](#)

[Diagram](#) | [Template](#)

[+ Add trigger](#) [+ Add destination](#)

Description
-

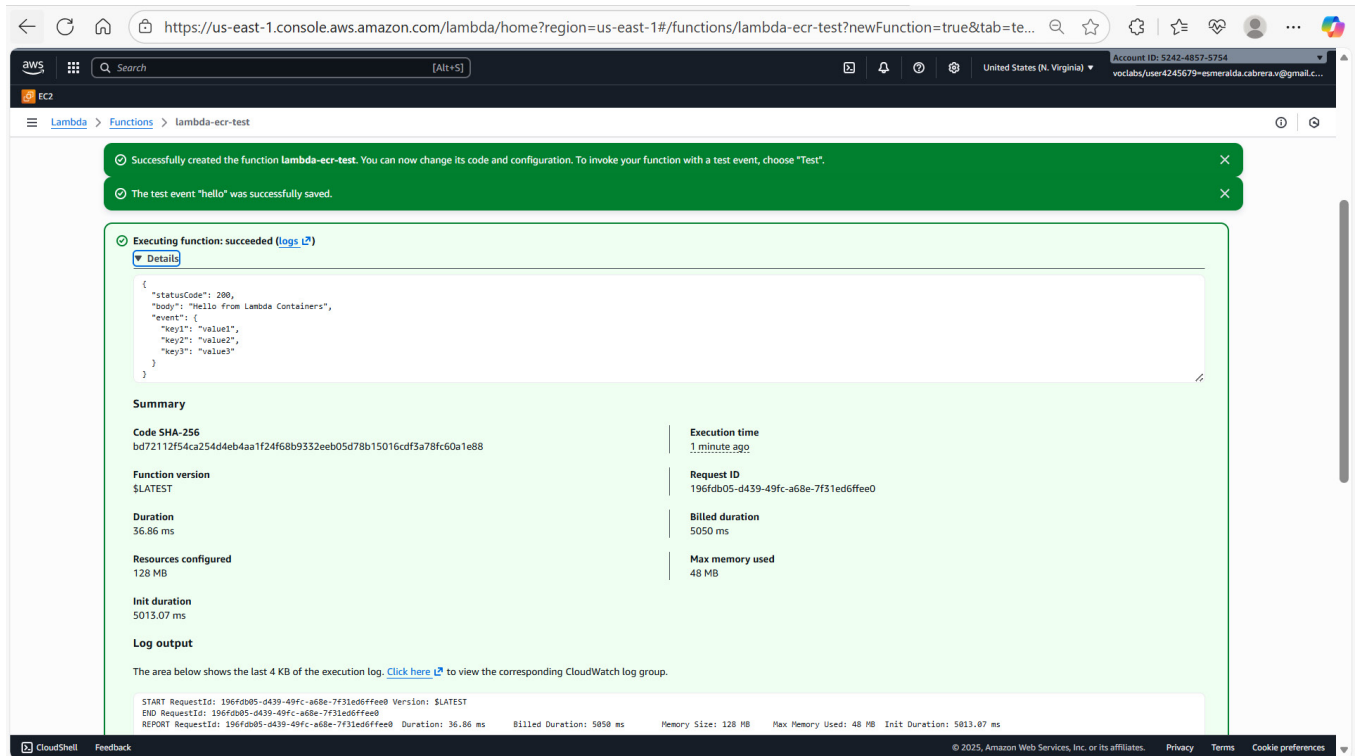
Last modified
1 minute ago

Function ARN
[arn:aws:lambda:us-east-1:524248575754:function:lambda-ecr-test](#)

Function URL [Info](#)

[Image](#) | [Test](#) | [Monitor](#) | [Configuration](#) | [Aliases](#) | [Versions](#)

Lambda Test Successful



Validation and Outcome

- The container image was built and stored successfully.
- Lambda pulled and executed the image without errors.
- The test invocation confirmed correct runtime behavior.

This confirms that container-based workloads can be executed serverlessly using AWS Lambda without managing application servers.

Why This Project Matters

This project reflects a modern cloud-native deployment pattern used in production environments where teams want:

- Predictable, containerized builds
- Serverless scaling and cost efficiency
- Minimal operational overhead

It demonstrates practical knowledge of AWS serverless + container integration, a highly relevant skill set for cloud engineering roles.